

متغیر `readCount` تعداد `readere` ها را ذخیره میکند. متغیر `source` نیز همان متغیر مشترکی است که `reader` ها سعی در خواندن آن دارند و `writer` ها نیز سعی در عوض کردن مقدار آن دارند. دو متغیر `source` , `readCount` بین بروسس ها به صورت مشترک استفاده میشوند. ممکن است `race` بین بروسس ها برای این دو متغیر به وجود آید. یک سمافور به نام `readSem` برای محافظت از متغیر `readCount` و یک سمافور به نام `sourceSem` برای محافظت از متغیر `source` تعریف میکنیم. تابع `function_for_writing` آیدی بروسس را میگیرد. بروسس تابع `sem_wait(sourceSem)` را صدا میزند تا بررسی کند آیا `writer` یا `reader` دیگری در حال کار با متغیر `source` میباشد یا خیر. اگر کسی در حال استفاده باشد؛ بروسس در صف خود را قرار میدهد تا بعدا بروسس دیگری آن را بیدار کند. در غیر این صورت از `sem_wait(sourceSem)` رد میشود. `writer` یک عدد رندم بین یک تا ده را به `source` اضافه میکند. سپس یک زمان رندم بین ۱ تا ۳ ثانیه `sleep` میکند. سپس تابع `sem_post(sourceSem)` را صدا میزند تا اگر بروسسی در صف هست؛ بیدار شود و به متغیر `source` دسترسی یابد. سپس یک زمان رندم بین ۱ تا ۳ ثانیه `sleep` میکند و دوباره این موارد را انجام میدهد زیرا در یک حلقه بی نهایت است. در تابع `function_for_reading` آیدی بروسس را به عنوان ورودی میدهم. خواننده ابتدا تابع `sem_wait(readSem)` را صدا میزند زیرا میخواهد با متغیر `readCount` کار کند. یک واحد به این متغیر اضافه میکند. اگر این بروسس اولین `reader` نباشد نیازی به صدا زدن `sem_wait(sourceSem)` نیست زیرا در سوال گفته شده چند خواننده به صورت همزمان حق دارند مقدار `source` را بخوانند. اما اگر بروسس اولین خواننده باشد؛ باید تابع `sem_wait(sourceSem)` را صدا بزند تا بررسی کند آیا قبل از او `writer` وارد `critical section` شده یا خیر. اگر `writer` وجود داشته باشد؛ این بروسس در صف قرار میگیرد تا بعدا توسط یک بروسس صدا زدهش شود. پس از این مراحل بروسس مقدار `source` را خوانده و یک عدد رندم بین ۱ تا ۳ `sleep` میکند. سپس مقدار `readCount` را یک واحد کم میکند. برای این کار نیز لازم است `sem_wait(readSem)` را صدا بزند. اگر بروسس فهمید هیچ خواننده دیگری وجود ندارد؛ تابع `sem_post(sourceSem)` را صدا میزند تا یک بروسس دیگر را بیدار کند تا مقدار `source` را به روز کند. در تابع `main` ابتدا یک حافظه اشتراکی بدون نام تشکیل میدهم. اندازه این حافظه اشتراکی برابر سایز دو تا سمافور و سایز دو تا عدد صحیح میباشد. سمافور ها را با مقدار یک مقدار دهی میکنیم تا به صورت `lock` عمل کنند. تعداد نویسنده ها و خواننده ها را ورودی میگیریم. به ازای تعداد خواننده ها و نویسنده ها `fork` میکنیم. برای بروسس های خواننده تابع `function_for_reading` و برای بروسس های نویسنده تابع `function_for_writing` را صدا میزنیم. برای این که تابع `main` تمام نشود در آخر `wait` قرار میدهم. با توجه به این فرزندان در حال اجرای یک تابع با حلقه بی نهایت هستند هیچ گاه کارشان تمام نمیشود و تابع `main` برای همیشه در این خط منتظر فرزندان میماند. عکس اجرای برنامه در فایل `question2_pictures.pdf` قرار داده شده.